NOTEPAD

Offline-First Intelligent Note-Taking Application

Project Synopsis & Technical Architecture

A consolidated, mobile-readable document describing the application's purpose, implemented capabilities, architecture, data flows, technologies, and implementation boundaries.

DOCUMENT CONTENTS

01 Project Synopsis

02 Technical Architecture Overview

03 Architecture Diagrams

Prepared for technical review by software engineers, technical leads, engineering managers, and prospective project supervisors.

Offline-First Intelligent Note-Taking Application

Project Overview

Notepad is a Flutter and Dart note-taking application organized around feature-oriented UI code and shared core services. The implemented application combines a Quill-based rich-text editor, encrypted local persistence, local keyword and semantic search, voice-controlled formatting, and on-demand Google Drive backup and restoration. Its local data paths allow note editing, indexing, and semantic maintenance to operate without requiring a cloud service; Groq is used for a remote voice-command parsing path when the local parser cannot resolve a command.

Problem Statement

The project addresses the need to capture and organize textual notes while preserving rich formatting, supporting fast retrieval, and reducing dependence on a continuously available remote data service. The repository implements document editing, search, note deletion and restoration, local semantic topic discovery, and backup workflows in one mobile-oriented workspace. Claims about user studies, measured performance improvements, deployment scale, or production reliability are not established from the implementation.

Key Features and Technical Highlights

- Rich-text editing uses **flutter_quill**. **NotePage** creates a **QuillController**, restores a document from stored rich-content Delta data through **document_delta.dart**, and observes editor changes through **NoteDataController** for debounced autosave.
- **NotesSection** is persisted in encrypted Hive boxes by **StorageService** using **HiveAesCipher**. The encryption key is generated with Hive and stored as Base64URL data through **SecureKeyVaultService** and **flutter_secure_storage**.
- Keyword search uses the SQLite FTS5 virtual table **notes_fts** in **SqliteFtsService**. **SearchService** combines FTS5 results with fuzzy matching and an in-memory fallback, then resolves result IDs to live note objects.
- Semantic topic discovery is implemented locally with **OnnxEmbeddingEngine**, **bert_tokenizer**, and the **bge-small-en-v1.5.onnx** model. Chunk embeddings and taxonomy vectors are stored in the separate persistent SQLite database **vectors.db** by **VectorStorageService**.
- Voice input uses **speech_to_text**. **VoiceCommandOrchestrator** first attempts **LocalVoiceParser**; unresolved commands can be sent to **GroqService**, which requests structured JSON instructions. **VoiceFormattingService** and **VoiceFormattingTargetResolver** apply the resulting operations locally to the **QuillController**.
- Google Drive synchronization uses Google Sign-In, the Drive API, and the **drive.appdata** scope. Active notes are serialized as backup JSON and uploaded to **appDataFolder**; the implementation retains three recent backup files and restores only note IDs absent from the local repository.

Technology Stack

The central technologies are Flutter and Dart, **flutter_quill**, Hive, **flutter_secure_storage**, SQLite FTS5, ONNX Runtime, BGE-small-en-v1.5, **speech_to_text**, the Groq API, Google Sign-In, and the Google Drive API. The application uses **ChangeNotifier**, **ValueNotifier**, **InheritedWidget**, **FutureBuilder**, and **ListenableBuilder**; no separate Provider, Riverpod, or BLoC package is established in the dependency configuration.

Architecture and Engineering Approach

The repository is divided into **lib/features** areas such as note, search, home, and trash, with shared database, bootstrap, theme, service, and constant code under **lib/core**. Controllers coordinate UI state, services implement feature operations, and repositories manage note and settings access, although the implementation also uses static service facades and singleton/module-level objects rather than a formal dependency-injection container. **AppInitializer** loads environment configuration, initializes Hive and encrypted storage, loads repositories, configures the composite search indexer, and starts semantic maintenance when applicable.

Hive is the local document store and the in-memory note cache is used to hydrate UI results. SQLite search and vector data are derived representations. Search failures are handled by returning empty results or falling back to other search modes; the code does not provide a general Hive text-search fallback for an FTS failure. Google Drive backup data is UTF-8 JSON at the application layer. No application-level encryption, hashing, compression, or cryptographic wrapping is applied before upload; therefore, Drive backups must not be described as application-encrypted.

Project Status

The repository contains implemented application code and tests for database services, repositories, search, semantic indexing, editor widgets, controllers, voice commands, backup analysis, and authentication behavior. The repository contains 119 Dart test files at the time of inspection. The README separately reports **340+** tests, approximately **73%** coverage, and **18k+** lines of application code; those figures were not independently verified from generated test, coverage, or code-volume reports. Exact runtime performance, deployment status, scalability, and production-readiness are not established from the implementation.

Notepad Architecture Overview

1. Architecture Introduction

Notepad is a feature-oriented Flutter application with a local-first data architecture. Notes are persisted in encrypted Hive boxes and represented in memory by **NoteRepository**. SQLite provides derived keyword and semantic search data through **notes_fts** and **vectors.db**. Separate services coordinate search, voice processing, and Google Drive backup. The implementation uses controllers, services, repositories, static facades, and singleton or module-level objects; no formal dependency-injection container is established.

2. Application Architecture and Persistence

Startup uses **AppInitializer** and **AppStartupBuilder**. **AppInitializer** loads environment configuration, initializes Hive through **Hive.initFlutter()**, mounts encrypted storage through **StorageService.initializeEncryptedStorage()**, loads application settings, initializes **NoteRepository**, and configures the composite search indexer. **AppStartupBuilder** exposes initialization state to startup UI and provides retry behavior for bootstrap failures.

NotePage owns the editor lifecycle. It creates a **QuillController** and restores an existing **NotesSection** by decoding its stored rich-content Delta through **document_delta.dart**. It also owns title, scroll, voice, and toolbar controllers, plus lifecycle callbacks that trigger background saving. The editor widget consumes the shared controller, so local formatting changes are rendered through the same document state.

NoteDataController observes title and document changes, debounces autosave, derives plain text and serialized rich content, and forwards note changes to **NoteRepository**. The editor signature includes the title and the serialized Quill Delta, allowing the controller to identify unchanged state. **NoteRepository** maintains active and deleted note collections, an ID-to-note cache, ordering information, and CRUD operations. It delegates persistent note operations to **StorageService** and search-index operations to the search indexer.

StorageService registers the generated Hive adapters for **NotesSection** and **AppSettings**, opens **notes_box** and **settings_box**, and exposes CRUD, export, import, and maintenance operations. **SecureKeyVaultService** obtains or creates the Hive encryption key through **flutter_secure_storage**; **StorageService** supplies it to **HiveAesCipher**. Hive is therefore the authoritative local persistence path in the inspected flows.

3. Search Architecture

Search is coordinated by **SearchController** and **SearchService**. **SearchController** debounces query changes and maintains query, date-filter, topic, loading, and result state. **SearchService** resolves candidate IDs through **NoteRepository.liveCacheMap**, so the UI receives hydrated **NotesSection** objects rather than document content loaded from a search database.

Keyword retrieval is implemented by **SqliteFtsService**. Its schema creates **notes_fts** as a genuine SQLite FTS5 virtual table with **id** and **updated_at** marked **UNINDEXED**, Unicode tokenization, diacritic removal, hyphen token support, and prefix indexes. The service sanitizes user input, constructs an FTS5 **MATCH** expression, and orders results using BM25 relevance followed by update time. The indexed title and plain-content fields are derived from notes.

When keyword retrieval produces no IDs, **SearchService** invokes **FuzzySearchService**, followed by an in-memory title/content fallback if necessary. A selected semantic topic is resolved through **SemanticSearchService**, **TopicDiscoveryService**, and the persistent vector database. **OnnxEmbeddingEngine** uses BGE-small-en-v1.5, BERT tokenization, and ONNX Runtime to generate normalized document vectors. **VectorStorageService** stores note chunks in **note_embeddings** and taxonomy vectors in **taxonomy_vectors** within **vectors.db**.

The implemented search sequence is: topic resolution, if selected; FTS5 retrieval; fuzzy fallback; in-memory fallback; candidate-ID combination; date and deletion filters; and hydration through

liveCacheMap. When both keyword and topic criteria are present, **SearchService** intersects their ID sets. Date-only searches use the live cache. Neither SQLite store replaces Hive as the document persistence source.

4. Voice Processing Architecture

Voice input begins with **speech_to_text** and is coordinated by **NoteVoiceController**. The resulting transcript goes to **VoiceCommandOrchestrator**, which first invokes **LocalVoiceParser** and falls back to **GroqService** when local interpretation is insufficient.

GroqService sends the transcript to Groq's Chat Completions endpoint and requests JSON containing **instructions**. The request contains the system prompt and transcript, not the note Delta, selection text, metadata, or complete editor state. Both paths produce structured **VoiceInstruction** data. **VoiceFormattingTargetResolver** resolves selections, whole-document targets, lines, paragraphs, literal phrases, ordinals, and occurrences against the local Quill document; **VoiceFormattingService** applies the operations to the local **QuillController**. Groq produces instructions, while mutation remains local.

5. Backup and Restoration

SyncController requests an active-note export from **NoteRepository** and **StorageService**. The JSON contains backup metadata and serialized active **NotesSection** records. **GoogleDriveService** authenticates with Google Sign-In, creates a Drive API client using the **https://www.googleapis.com/auth/drive.appdata** scope, and uploads UTF-8 JSON to Drive's **appDataFolder**. Backup files use the **notepad_backup_** prefix, and cleanup retains the three newest files.

Restore lists files in **appDataFolder**, downloads media, decodes it as UTF-8, and passes the JSON to repository import logic. **BackupSyncService** skips existing IDs and imports absent IDs; it does not perform newer-version merging or overwrite local notes. No application-level encryption, hashing, compression, or cryptographic wrapping is applied before upload, and restore has no decryption step. The Drive payload must not be described as application-encrypted.

6. Technology Stack

Component	Technology
Mobile application	Flutter / Dart
Rich-text editor	**flutter_quill**
Local persistence	Hive CE
Local key storage	**flutter_secure_storage**
Keyword search	SQLite FTS5 / **sqflite**
Semantic inference	ONNX Runtime / BGE-small-en-v1.5
Voice input	**speech_to_text**
AI fallback	Groq Chat Completions API / **http**
Cloud authentication and backup	Google Sign-In / Google Drive API

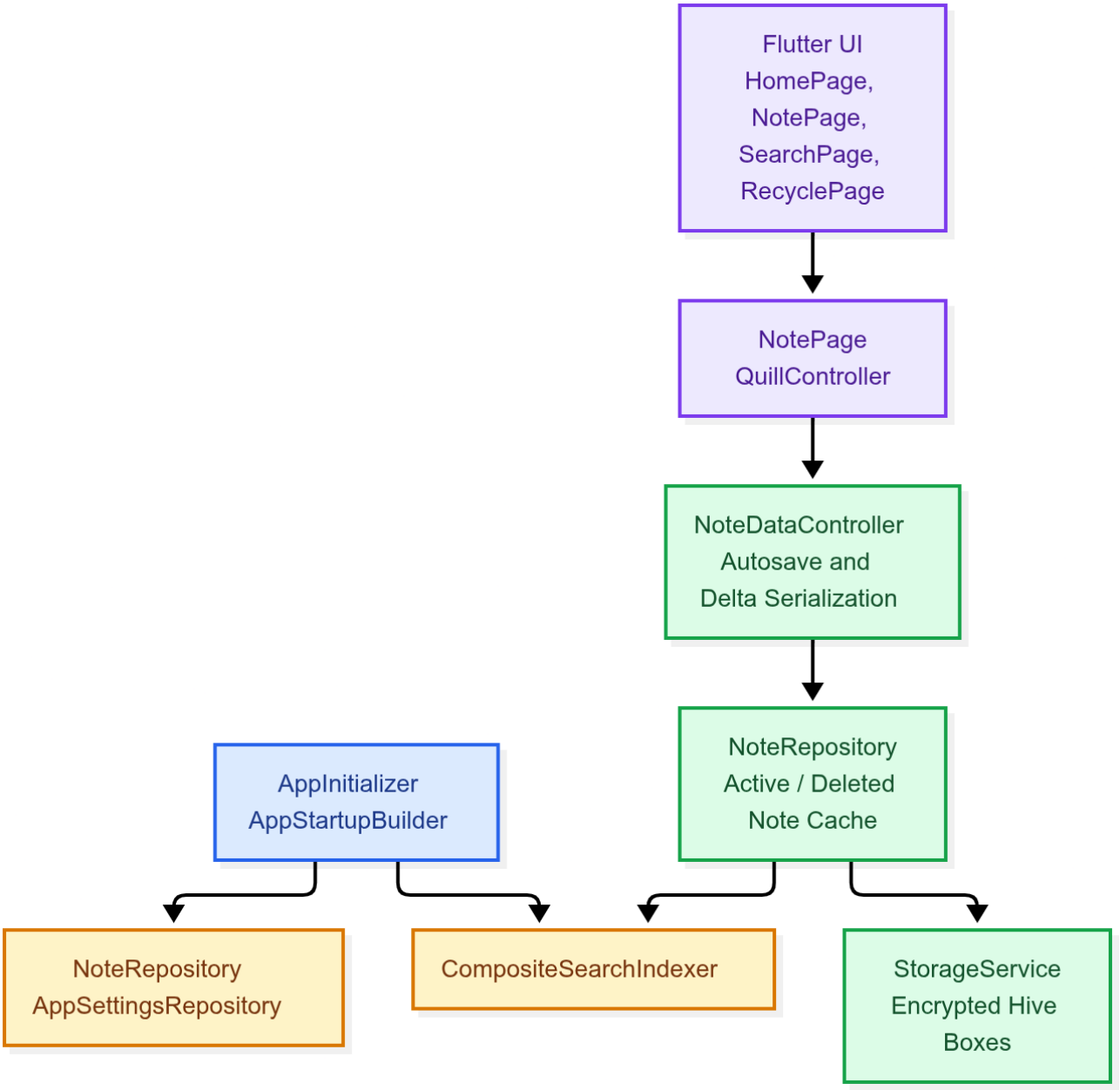
7. Implementation Decisions and Limitations

Area	Confirmed implementation detail
Persistence	Hive boxes are opened with **HiveAesCipher**.

Area	Confirmed implementation detail
Key management	**SecureKeyVaultService** stores the Base64URL-encoded key through **flutter_secure_storage**.
Search indexing	**notes_fts** is a derived SQLite FTS5 virtual table.
Semantic storage	**vectors.db** stores derived note and taxonomy vectors.
Search results	Candidate IDs are hydrated through **NoteRepository.liveCacheMap**.
Voice processing	**LocalVoiceParser** is attempted before Groq fallback.
Cloud backup	Active-note JSON is uploaded without application-level encryption.
Dependency management	Direct construction, static facades, and singleton/module-level objects are used; no formal DI container is established.

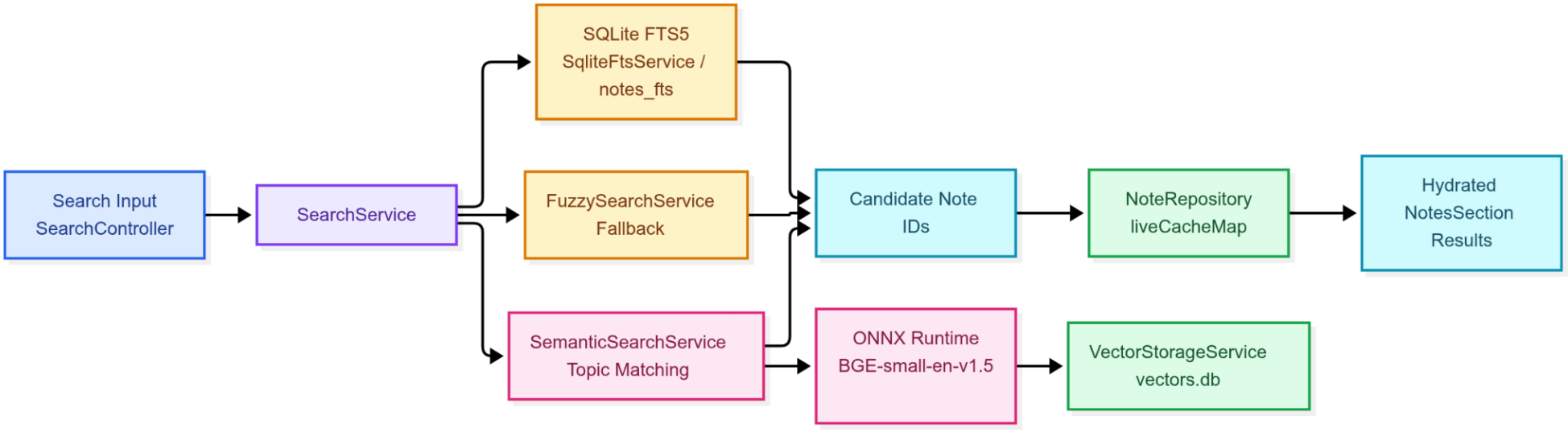
The repository demonstrates local persistence, derived search indexes, local semantic processing, and structured voice formatting, but does not establish scalability, measured performance, deployment status, or production readiness. It contains 119 Dart test files. The README reports **340+** tests, approximately **73%** coverage, and **18k+** lines of application code; those figures were not independently verified from generated reports.

Application Architecture and Persistence



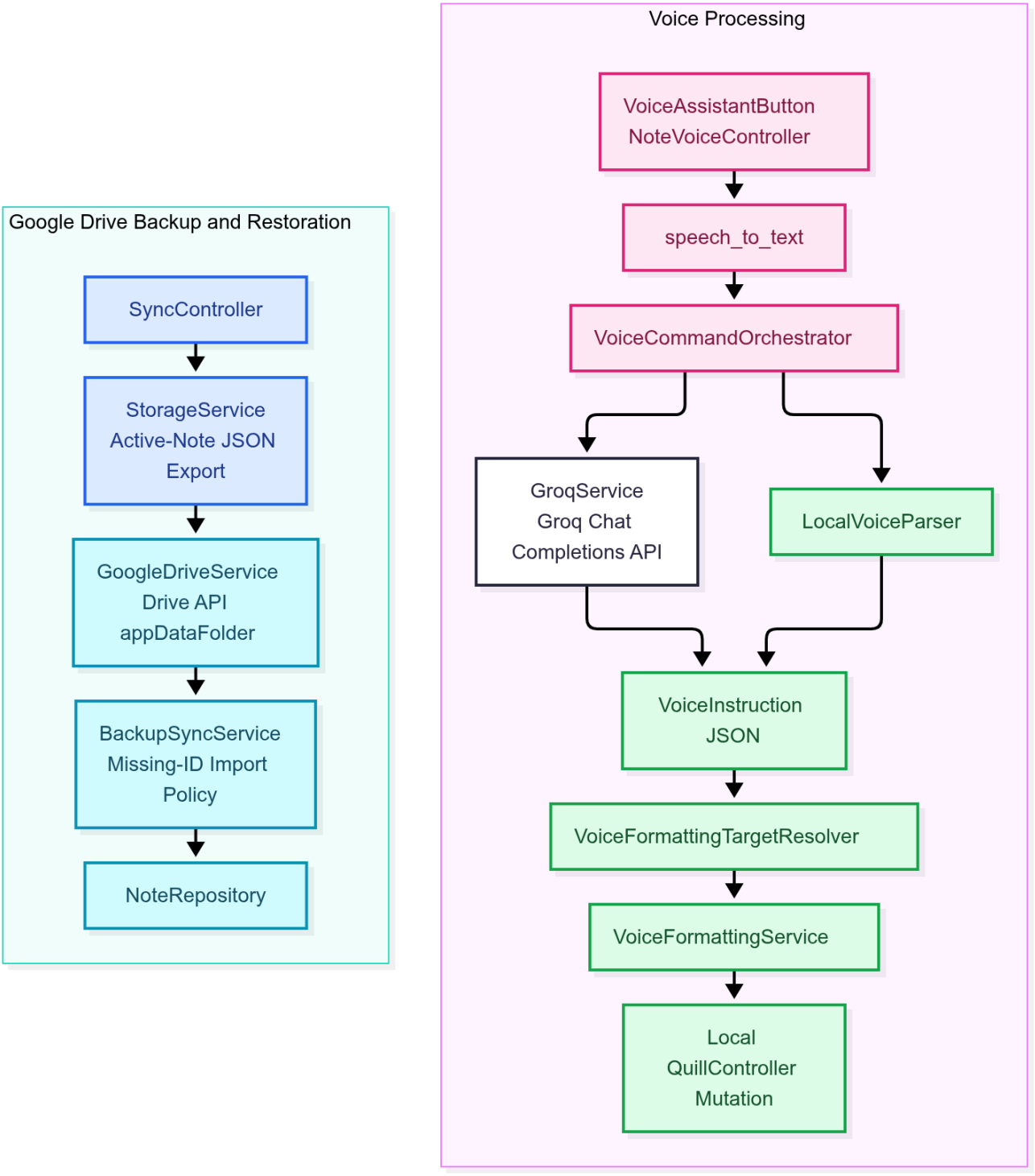
Rendered architecture diagram based on the project's documented component relationships.

Search Architecture



Rendered architecture diagram based on the project's documented component relationships.

Voice Processing and Cloud Backup



Rendered architecture diagram based on the project's documented component relationships.